Version Control Guidelines

Shinsato, Miles

Bellevue University, CSD-380

Assignment 3.2

November 7, 2026

Version Control Guidelines

Introduction

Version control is a crucial piece of the DevOps mindset. Version control is a practice of tracking and recording any changes made through a software's lifecycle. It also allows developers to compare versions, supports collaboration, and maintains a history of the changes made. GitLab defined it as: "an important software development practice for tracking and managing changes made to code and other files. It is closely related to source code management" (GitLab, n.d.). There are many ways that version control can be implemented and what is prioritized. I researched a few opinions on version control and the guidelines that are prioritized. By no means will this be exhaustive of what each source's version control guidelines are. However, it will be more of an overview of each source instead that covers what I would consider the most important points, what is found in common, as well as a standout that each source has.

Atlassian

Atlassian is an Australian software company that is often filled with great information. When they wrote of source code management, which does have many similarities to version control, they had several best practices they pointed towards. Here are a few of the ones that I want to highlight. Atlassian spoke of committing often. Think of every commit as if making checkpoints throughout the developing process to revert to in case of a mistake or error. The more frequently commits are made, the more opportunities developers have to revert changes or recover previous work. They also make it a point to verify any work is being done on the latest global version. This is crucial since it can help reduce compatibility and other types of errors that could arise later. What I would consider the one of the most crucial guidelines they have is to

make detailed notes with every commit. Having detailed notes can give a snapshot of the commit being made and give any important context that could be lost to time if it is not documented. The purpose that Atlassian points out is “these log messages become the canonical history of the project’s development and leave a trail for future contributors to review” (Atlassian, n.d.-a). This is the equivalent of journalling what is important to remember from each commit with their own entry.

Nilsen

An article on Dev.to written by full-stack developer, Pelle Nilsen had a few version control practices that they felt important to focus in on. They shared a few with Atlassian such as, making commits often and writing detailed message to go with each commit. However, they also added a few that I felt important to bring up. The first that I want to highlight is using .gitignore to exclude any files that do not need to be tracked by version control. Nilsen points out that these files could be things like build artifacts or dependencies. (Nilsen, 2024). The other highlight I read in this article was using tags for releases. This can easily help narrow down any issues across versions and attempt to replicate. Additionally, it can be a major tool when attempting to troubleshoot any bugs that can be due to a specific code change in a specific version.

Ernst

Professor Michael Ernst wrote an article regarding the idea of version control and what they considered as version control best practices. Similar to the two sources, Ernst emphasized the importance of descriptive commit messages. However, they did have a tip that neither of the others had, he described it as: make each commit a logical unit. This is referring to each time a commit is made, it is for a specific and articulable reason. The example that was given described a bug fix, new feature, and typo fix as a reason for three separate commits instead of combining

them all into one. The reasoning behind this being that it "makes it easier to locate the changes related to some particular feature or bug fix … to determine the changes responsible for buggy behavior" (Ernst, 2024). If each commit was tied to a singular reason, it becomes much easier to identify what changes may have introduced a problem when troubleshooting.

Comparing the Guidelines

When comparing these three sources, there were a few similarities that stood out. All three emphasized the importance of descriptive commit messages. Atlassian and Nilsen both highlighted that committing frequently throughout the development process should be done. Ernst added another layer by suggesting that each commit should represent a single logical change instead of combining commits. While these recommendations are similar, they each provide a different benefit. Frequent commits allow for more options to revert changes, descriptive messages help developers understand what was changed, and commits after each logical change isolate problems when troubleshooting. There were also differences between each source. Atlassian provided a broader look at managing source code throughout development. Nilsen brought up the use .gitignore and tagging releases. Ernst focused more on the organization of individual commits.

Outdated Guidelines

While researching, I did not find any recommendations from these sources that I would consider truly outdated. However, some of these practices do appear to have evolved. One example is the use of long-lived development branches. In older workflows, developers might maintain separate branches with extended periods before merging their work. This could result in significant differences between branches, making integration and resolving conflicts more difficult. However, modern DevOps practices generally encourage smaller changes with more

frequent integration. While commits with many changes can work, it can create headaches when troubleshooting if something goes wrong. Smaller and more frequent changes reduce the likelihood of merge conflicts and allow for problems to be identified earlier. I would consider this more of an evolution of version control practices rather than an outdated guideline. The fundamental idea of maintaining a history of changes, organizing commits, and review work remain just as important today as it did in the past.

My Guidelines

However, I found a few common threads among a few different sources that I would implement into guidelines that I would prioritize. The most important is to make frequent commits for individual articulable changes to a project's code such as a bug fix or optimization. Keeping commits focused on individual changes instead of combining changes is to help make troubleshooting and reviewing changes easier. Additionally, I find it important to have a detailed description of what the changes to the code would be when the commit is made. Instead of simply saying "fixed a typo," the commit message could specify the typo that was fixed and in what line of the code it was present. If there ends up being an issue, the commit message can narrow down the search to a specific point, saving time and energy to address. Using tags to identify releases is another guideline I find value in. These can provide identifiable checkpoints for specific versions and can make it easier to investigate version-specific issues. Another guideline I would prioritize is using .gitignore to prevent unnecessary files from being tracked. Keeping a repository organized is important, especially when working with larger projects that can contain many generated files and dependencies. This can help keep the repository manageable and make it easier for developers to locate the files they actually need.

Conclusion

Ultimately, I believe that having consistent guidelines is just as important as the individual practices themselves. When everyone on a development team follows the same expectations for committing, documenting, and organizing changes, it makes collaboration easier and helps maintain the integrity of the project throughout its lifecycle.

References

Atlassian. (n.d.-a). *Source Code Management | Atlassian Git Tutorial*. Retrieved November 7, 2026, from https://www.atlassian.com/git/tutorials/source-code-management/

Atlassian. (n.d.-b). *What is version control | Atlassian Git Tutorial*. https://www.atlassian.com/git/tutorials/what-is-version-control

Ernst, Michael. (2024, April 8). *Version Control Concepts and Best Practices*. Retrieved November 7, 2026, from https://homes.cs.washington.edu/~mernst/advice/version-control.html

GitLab. (n.d.). *What is version control?* GitLab. https://about.gitlab.com/topics/version-control/

Nilsen, Pelle. (2024, September 17). *Version Control Best Practices*. DEV Community. Retrieved November 7, 2026, from https://dev.to/pellenilsen/version-control-best-practices-mastering-git-branching-strategies-and-collaborative-workflows-5gal